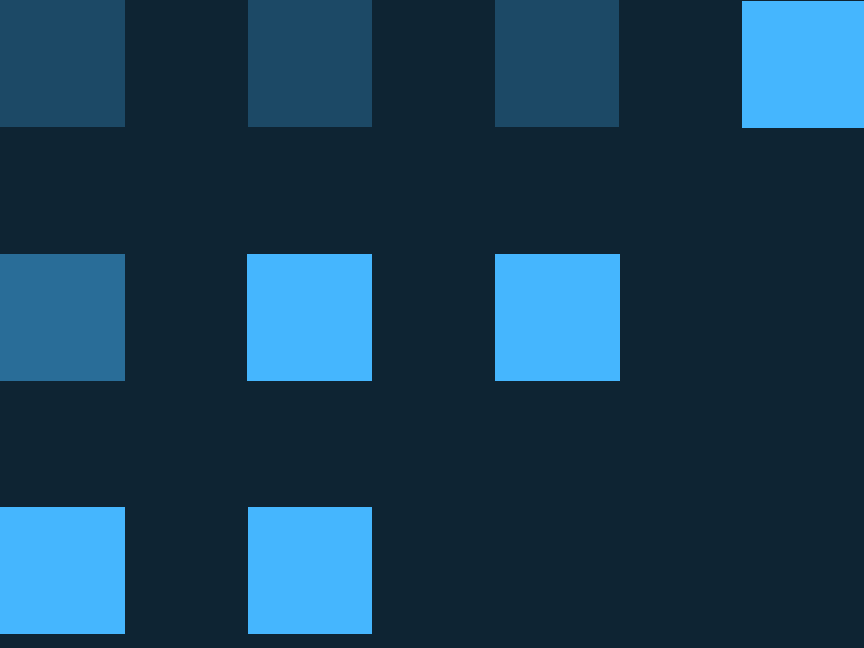




Title Twister

Jeu 2048



SOMMAIRE

01

TECHNOLOGIES

Outils utilisés pour réaliser le projet

02

DEVELOPPEMENT

Le code réalisé et la logique derrière

03

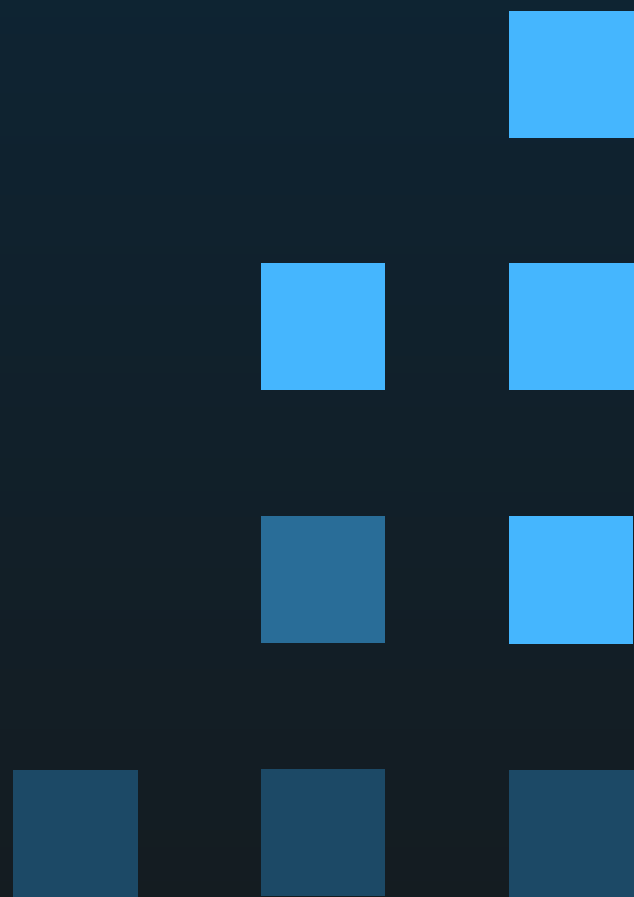
DEMONSTRATION

Démonstration d'une partie de jeu en direct

04

CONCLUSION

Le pistes d'améliorations et difficultés rencontrées



01

TECHNOLOGIES



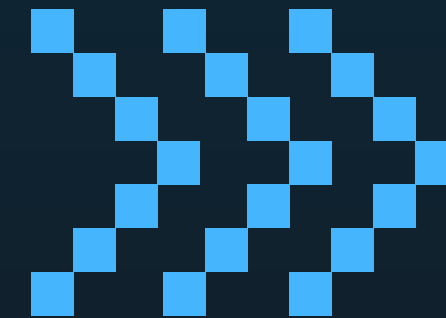
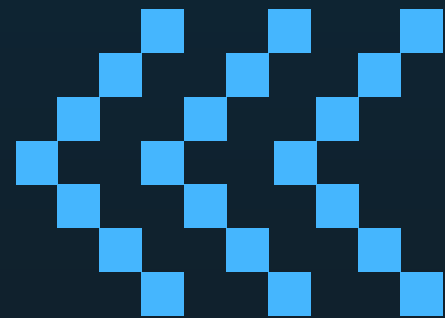
Ce projet consiste à développer le célèbre jeu 2048 en C++ avec une interface graphique réalisée grâce à SDL3. L'application permet de déplacer et fusionner les tuiles jusqu'à atteindre l'objectif choisi par le joueur. D'autres fonctionnalités ont été ajoutées comme un compteur de score en temps réel, la possibilité de recommencer une partie sans fermer la fenêtre, ainsi qu'un système de sélection de l'objectif (1024, 2048 ou 8192). Pour rendre l'expérience plus dynamique et ludique, une animation du personnage "Duck Skater" a également été ajoutée à l'interface.

—OBJECTIF DU PROJET

TECHNOLOGIES UTILISEES



SDL_TTF



SDL_IMAGE

02

DEVELOPPPEMENT



ARBORESCENCE DE FICHIERS

```
.
├── TileTwister/
│   ├── fonts/
│   │   └── PressStart2P-Regular.ttf
│   ├── images/
│   │   ├── defeat.png
│   │   ├── duck.png
│   │   └── victory.png
│   ├── include/
│   │   ├── Game.hpp
│   │   ├── GameObject.hpp
│   │   ├── Grid.hpp
│   │   └── Window.hpp
│   ├── src/
│   │   ├── Game.cpp
│   │   ├── GameObject.cpp
│   │   ├── Grid.cpp
│   │   ├── Window.cpp
│   │   └── main.cpp
│   ├── .gitignore
│   ├── Makefile
│   └── README.md
```

Classe Window



```
#ifndef WINDOW_HPP
#define WINDOW_HPP
#include <string>
#include <SDL3/SDL.h>
#include <SDL3_ttf/SDL_ttf.h>

// Class representing the game window
class Window {

private:

    std::string title;
    int width;
    int height;
    SDL_Window* pWindow;
    SDL_Renderer* pRenderer;

public:

    Window(std::string title, int width, int height);

    bool initialize();
    SDL_Renderer* getRenderer();
    SDL_Window* getWindow();

    ~Window();
};

#endif
```

Classe Window

```

#include "Window.hpp"
#include <iostream>
#include <string>
using namespace std;

// Constructor by initialisation list : Window
Window::Window(std::string title, int width, int height)
    : title(title), width(width), height(height), pWindow(nullptr), pRenderer(nullptr)
{
}

// Method for initialising the window
bool Window::initialize() {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        SDL_LogError(SDL_LOG_CATEGORY_APPLICATION, "[DEBUG] > %s", SDL_GetError());
        return false;
    }

    if (!TTF_Init()) {
        SDL_LogError(SDL_LOG_CATEGORY_APPLICATION, "[DEBUG] > %s", SDL_GetError());
        return EXIT_FAILURE;
    }

    pWindow = SDL_CreateWindow(this->title.c_str(), this->width, this->height, 0);
    if (pWindow == nullptr) {
        SDL_LogError(SDL_LOG_CATEGORY_APPLICATION, "[DEBUG] > %s", SDL_GetError());
        SDL_Quit();
        return false;
    }

    pRenderer = SDL_CreateRenderer(pWindow, NULL);
    if (pRenderer == nullptr) {
        SDL_LogError(SDL_LOG_CATEGORY_APPLICATION, "[DEBUG] > %s", SDL_GetError());
        SDL_Quit();
        return false;
    }
    return true;
}

```

```

// Getter to retrieve the pointer
SDL_Renderer* Window::getRenderer(){
    return this->pRenderer;
}

// Getter to retrieve the pointer
SDL_Window* Window::getWindow(){
    return this->pWindow;
}

// The destructor to free memory
Window::~~Window() {
    if (pRenderer != nullptr) {
        SDL_DestroyRenderer(pRenderer);
    }
    if (pWindow != nullptr) {
        SDL_DestroyWindow(pWindow);
    }
    TTF_Quit();
    SDL_Quit();
}

```

Classe GameObject

```

// Header file: GameObject.h
#ifndef GAMEOBJECT_HPP
#define GAMEOBJECT_HPP

#include <SDL3/SDL.h>
#include <SDL3_ttf/SDL_ttf.h>
#include <SDL3_image/SDL_image.h>
#include <string>

// An abstract class representing an element displayed in the graphical user interface
class GameObject {

protected:
    int coordinateX;
    int coordinateY;
    int sizeX;
    int sizeY;

public:

    // Constructor of the abstract class
    GameObject(int coordinateX, int coordinateY, int sizeX, int sizeY);

    virtual void draw(SDL_Renderer* pRenderer) = 0;
};
```

```

// Child class that inherits from the parent class
class Tile : public GameObject {

private:
    const SDL_Color& color;

public:

    Tile(int coordinateX, int coordinateY, int sizeX, int sizeY, const SDL_Color& color);

    void draw(SDL_Renderer* pRenderer) override;
};

// Child class that inherits from the parent class
class Image : public GameObject {

public:

    using GameObject::draw;

    Image(int coordinateX, int coordinateY, int sizeX, int sizeY);

    void draw(SDL_Renderer* pRenderer) override;
    void draw(SDL_Renderer* pRenderer, std::string path); // overload
    int getCoordinateX();
    int getSizeX();
    void setCoordinateX(int value);
};

#endif
```

Classe GameObject

```
#include "GameObject.hpp"
#include <iostream>
using namespace std;

// Constructor by initialisation list : GameObject
GameObject::GameObject(int coordinateX, int coordinateY, int sizeX, int sizeY)
    : coordinateX(coordinateX), coordinateY(coordinateY), sizeX(sizeX), sizeY(sizeY)
{
}

// Constructor by initialisation list : Tile
Tile::Tile(int coordinateX, int coordinateY, int sizeX, int sizeY, const SDL_Color& color)
    : GameObject(coordinateX, coordinateY, sizeX, sizeY),
    color(color)
{
}

// Method for drawing a tile
void Tile::draw(SDL_Renderer* pRenderer) {
    SDL_FRect tileGame = {(float)this->coordinateX, (float)this->coordinateY, (float)this->sizeX, (float)this->sizeY};
    SDL_SetRenderDrawColor(pRenderer, this->color.r, this->color.g, this->color.b, this->color.a);
    SDL_RenderFillRect(pRenderer, &tileGame);
}
```

```
// Constructor by initialisation list : Image
Image::Image(int coordinateX, int coordinateY, int sizeX, int sizeY)
    : GameObject(coordinateX, coordinateY, sizeX, sizeY)
{
}

void Image::draw(SDL_Renderer* pRenderer){
    return;
}

// Overload method for displaying an image, as we need the path
void Image::draw(SDL_Renderer* pRenderer, std::string path) {
    SDL_Texture* texture = IMG_LoadTexture(pRenderer, path.c_str());
    SDL_FRect destination = {(float)this->coordinateX, (float)this->coordinateY, (float)this->sizeX, (float)this->sizeY};
    SDL_RenderTexture(pRenderer, texture, nullptr, &destination);
    SDL_DestroyTexture(texture);
}

int Image::getCoordinateX() {
    return this->coordinateX;
}

int Image::getSizeX() {
    return this->sizeX;
}

void Image::setCoordinateX(int value) {
    this->coordinateX = value;
}
```

Classe Grid

```
#ifndef GRID_HPP
#define GRID_HPP
#include <utility>
#include <SDL3/SDL.h>
#include <SDL3_ttf/SDL_ttf.h>
#include <string>
#include <random>
#include <vector>
#include <iostream>

// Class representing the game grid as a two-dimensional vector
class Grid {

private:

    std::vector<std::vector<int>>> grid;
    TTF_Font* font;
    std::mt19937 gen;
    int score;
    int target;

public:

    Grid();

    void initialize();
    void draw(SDL_Renderer* pRenderer);
    void drawText(SDL_Renderer* pRenderer, std::string text, int x, int y, const SDL_Color& color);
    void drawChoice(SDL_Renderer* pRenderer);
    void addTile();
    void shift(char direction);
    void merge(char direction);
    bool move(char direction);
    bool loadFont();
    bool findNumber(int number);
    bool shiftAvailable();
    void newGame();
    void setTarget(int value);
    int getTarget();
    int getScore();
    void setFontSize(int value);
    void choice(int x, int y);

    ~Grid();

};

#endif
```

Classe Grid

```
#include "Grid.hpp"
#include "GameObject.hpp"
#include <iostream>
#include <vector>
#include <utility>
#include <algorithm>
using namespace std;

// Constructor by initialisation list : Grid
Grid::Grid()
    : grid(4, std::vector<int>(4, 0)), font(nullptr), gen(std::random_device{}()),
    score(0), target(0)
{
}

// Method for loading the font contained in the "fonts" folder
bool Grid::loadFont() {
    font = TTF_OpenFont("fonts/PressStart2P-Regular.ttf", 20);
    return font != nullptr;
}

// Method for initialising the grid by filling it with zeros and placing the two
// starting tiles
void Grid::initialize() {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            grid[i][j] = 0;
        }
    }
    addTile();
    addTile();
}
```

```
// Method for adding a random tile to the grid
void Grid::addTile() {
    std::uniform_int_distribution<int> distribPosition(0, 3); // A random number between 0 and 3 for the position
    std::uniform_int_distribution<int> distribValue(1, 10); // A random number to determine the tile's value (2
    or 4) based on probabilities

    int row;
    int column;

    do {
        row = distribPosition(gen);
        column = distribPosition(gen);
    } while (grid[row][column] != 0);

    // 90% chance of a 2 appearing and 10% chance of a 4 appearing
    if (distribValue(gen) <= 9) {
        grid[row][column] = 2;
    } else {
        grid[row][column] = 4;
    }
}

// Method for displaying text
void Grid::drawText(SDL_Renderer* pRenderer, std::string text, int x, int y, const SDL_Color& color) {
    SDL_SetRenderDrawColor(pRenderer, color.r, color.g, color.b, color.a);
    SDL_Surface* surface = TTF_RenderText_Blended(font, text.c_str(), 0, color);
    if (surface == nullptr) {
        return;
    }

    SDL_Texture* texture = SDL_CreateTextureFromSurface(pRenderer, surface);
    if (texture == nullptr) {
        SDL_DestroySurface(surface);
        return;
    }

    // The number is centred relative to the tile's size
    float texteX = x + (90 - surface->w) / 2.0f;
    float texteY = y + (90 - surface->h) / 2.0f;

    SDL_FRect destination = {texteX, texteY, (float)surface->w, (float)surface->h};
    SDL_RenderTexture(pRenderer, texture, nullptr, &destination);

    SDL_DestroyTexture(texture);
    SDL_DestroySurface(surface);
}
```

Classe Grid

```
// Method for displaying the grid
void Grid::draw(SDL_Renderer* pRenderer) {

    //The live score is displayed
    drawText(pRenderer, "Score : " + std::to_string(this->score), 90, 30, { 255, 255, 0, 255 });
    drawText(pRenderer, "Objectif : ", 340, 30, { 255, 255, 0, 255 });
    SDL_Color color = { 0, 0, 0, 255 };
    if(target == 1024) {
        color = { 220, 80, 120, 255 };
    } else if (target == 2048) {
        color = { 180, 60, 160, 255 };
    } else {
        color = { 100, 40, 220, 255 };
    }
    Tile tileTarget(460, 25, 90, 90, color);
    tileTarget.draw(pRenderer);
    drawText(pRenderer, std::to_string(this->target), 460, 25, { 0, 0, 0, 255 });

    // We create the background of the game board
    Tile boardGame(75, 125, 450, 450, { 28, 73, 102, 255 });
    boardGame.draw(pRenderer);
```

```
// We go through our 16 cell
for (int i = 0; i < 4; ++i) {
    for (int j = 0; j < 4; ++j) {
        int x = 105 + j * 100;
        int y = 155 + i * 100;

        SDL_Color color = { 41, 109, 152, 255 };

        // The colour of the tile is determined by the value in the cell
        switch(grid[i][j]) {
            case 2:
                color = { 238, 228, 218, 255 };
                break;
            case 4:
                color = { 232, 220, 200, 255 };
                break;
            case 8:
                color = { 255, 183, 120, 255 };
                break;
            case 16:
                color = { 255, 150, 90, 255 };
                break;
            case 32:
                color = { 255, 120, 80, 255 };
                break;
            case 64:
                color = { 255, 90, 60, 255 };
                break;
            case 128:
                color = { 240, 180, 70, 255 };
                break;
            case 256:
                color = { 255, 160, 50, 255 };
                break;
            case 512:
                color = { 255, 130, 40, 255 };
                break;
            case 1024:
                color = { 220, 80, 120, 255 };
                break;
            case 2048:
                color = { 180, 60, 160, 255 };
                break;
            case 4096:
                color = { 140, 50, 200, 255 };
                break;
            case 8192:
                color = { 100, 40, 220, 255 };
                break;
            default:
                break;
        }

        // For each cell in the grid, a tile is created and displayed
        Tile tileGame(x, y, 90, 90, color);
        tileGame.draw(pRenderer);
        // If the cell contains a value, it is displayed on top of the tile
        if (grid[i][j] != 0) {
            drawText(pRenderer, std::to_string(grid[i][j]), x, y, { 0, 0, 0, 255 });
        }
    }
}
}
```

Classe Grid

```
// Method for the opening screen to select the tile to reach in order to
win the game
void Grid::drawChoice(SDL_Renderer* pRenderer) {
    drawText(pRenderer, "Votre objectif", 260, 30, { 255, 255, 0, 255 });
    int choices[] = { 1024, 2048, 8192 };
    SDL_Color colors[] = {{ 69, 182, 254, 255 }, { 41, 109, 152, 255 }, {
28, 73, 102, 255 }};
    for(int i = 0 ; i < 3 ; i++) {
        Tile choice(240, (i + 1) * 150, 120, 120, colors[i]);
        choice.draw(pRenderer);
        drawText(pRenderer, std::to_string(choices[i]), 255, (i + 1) *
150 + 15, { 255, 255, 255, 255 });
    }
}
```

```
// Method for managing movements by sorting the grid
void Grid::shift(char direction) {
    switch(direction) {
        // All the numbers in the row are shifted to the left
        case 'l':
            for(int i = 0 ; i < 4 ; i++) {
                stable_partition(grid[i].begin(), grid[i].end(), [](int n) {
                    return n != 0;
                });
            }
            break;

        // All the numbers in the row are shifted to the right
        case 'r':
            for(int i = 0 ; i < 4 ; i++) {
                stable_partition(grid[i].begin(), grid[i].end(), [](int n) {
                    return n == 0;
                });
            }
            break;

        // The numbers in the column are grouped together before being sorted from left to right
        case 'u':
            for(int i = 0 ; i < 4 ; i++) {
                std::vector<int> temp(4);
                for(int j = 0 ; j < 4 ; j++) {
                    temp[j] = grid[j][i];
                }
                stable_partition(temp.begin(), temp.end(), [](int n) {
                    return n != 0;
                });
                for(int j = 0 ; j < 4 ; j++) {
                    grid[j][i] = temp[j];
                }
            }
            break;

        // The numbers in the column are grouped together before being sorted from right to left
        case 'd':
            for(int i = 0 ; i < 4 ; i++) {
                std::vector<int> temp(4);
                for(int j = 0 ; j < 4 ; j++) {
                    temp[j] = grid[j][i];
                }
                stable_partition(temp.begin(), temp.end(), [](int n) {
                    return n == 0;
                });
                for(int j = 0 ; j < 4 ; j++) {
                    grid[j][i] = temp[j];
                }
            }
            break;
    }
}
```

Classe Grid

```
// Method for calculating the merging of two tiles along a given direction
void Grid::merge(char direction) {
    switch(direction) {
        // We scan the line from left to right
        case 'l':
            for(int i = 0 ; i < 4 ; i++) {
                for(int j = 0 ; j < 3 ; j++) {
                    if(grid[i][j] == grid[i][j+1]){
                        this->score += grid[i][j];
                        grid[i][j] *= 2;
                        grid[i][j+1] = 0;
                    }
                }
            }
            break;

        // We scan the line from right to left
        case 'r':
            for(int i = 0 ; i < 4 ; i++) {
                for(int j = 3 ; j > 0 ; j--) {
                    if(grid[i][j] == grid[i][j-1]){
                        this->score += grid[i][j];
                        grid[i][j] *= 2;
                        grid[i][j-1] = 0;
                    }
                }
            }
            break;

        // We scan the column from left to right
        case 'u':
            for(int i = 0 ; i < 4 ; i++) {
                for(int j = 0 ; j < 3 ; j++) {
                    if(grid[j][i] == grid[j+1][i]){
                        this->score += grid[j][i];
                        grid[j][i] *= 2;
                        grid[j+1][i] = 0;
                    }
                }
            }
            break;

        // We scan the column from right to left
        case 'd':
            for(int i = 0 ; i < 4 ; i++) {
                for(int j = 3 ; j > 0 ; j--) {
                    if(grid[j][i] == grid[j-1][i]){
                        this->score += grid[j][i];
                        grid[j][i] *= 2;
                        grid[j-1][i] = 0;
                    }
                }
            }
            break;
    }
}
```

```
// Method for managing correct movement and merging according to direction
bool Grid::move(char direction) {
    auto oldGrid = grid;
    shift(direction);
    merge(direction);
    shift(direction); // Move again after a merge
    return oldGrid != grid; // We check if the grid has changed and therefore if we need to add
    a new tile or not
}

// Method for searching for a specific value in the grid (used for 0 and 2048 in particular)
bool Grid::findNumber(int number) {
    for(int i = 0 ; i < 4 ; i++) {
        for(int j = 0 ; j < 4 ; j++) {
            if(grid[i][j] == number){
                return true;
            }
        }
    }
    return false;
}

// Method for checking if you can still make a move and therefore the game is not over
bool Grid::shiftAvailable() {
    for(int i = 0 ; i < 4 ; i++) {
        for(int j = 0 ; j < 3 ; j++) {
            if((grid[i][j] == grid[i][j+1]) || (grid[j][i] == grid[j+1][i])){
                return true;
            }
        }
    }
    return false;
}
```

Classe Grid

```
// Method for starting a new game
void Grid::newGame() {
    initialize();
    this->score = 0;
    setFontSize(20);
}

// Setter to change the target value
void Grid::setTarget(int value) {
    this->target = value;
}

// Setter to change the font's size
void Grid::setFontSize(int value) {
    float newSize = value;
    if (!TTF_SetFontSize(this->font, newSize)) {
        std::cerr << "Erreur lors du changement de taille: "
        << SDL_GetError() << std::endl;
    }
}
```

```
// Getter to retrieve the target value
int Grid::getTarget() {
    return this->target;
}

// Getter to retrieve the score value
int Grid::getScore() {
    return this->score;
}

// A method for determining, based on the position on the screen,
// what the objective is for that game
void Grid::choice(int x, int y) {
    if(x >= 240 && x <= 360 && y >= 150 && y <= 270) {
        this->target = 1024;
    } else if(x >= 240 && x <= 360 && y >= 300 && y <= 420) {
        this->target = 2048;
    } else if(x >= 240 && x <= 360 && y >= 450 && y <= 570) {
        this->target = 8192;
    }
}

// The destructor to free memory
Grid::~Grid() {
    if (font != nullptr) {
        TTF_CloseFont(font);
    }
}
```

Classe Game

```

    #ifndef GAME_HPP
    #define GAME_HPP

    #include "Grid.hpp"
    #include "GameObject.hpp"
    #include "Window.hpp"

    // Contains the grid and is responsible for displaying it and detecting keys pressed on the keyboard
    class Game {

    private:
        Grid grid;
        Image finalImage;
        bool targetSelected;
        bool gameInProgress;

    public:
        Game( );

        void initialize();
        void startInterface(SDL_Renderer* pRenderer);
        void manageKeyboard(SDL_Event& event);

    };

    #endif
```

Classe Game

```
#include "Game.hpp"
#include <iostream>
#include <string>
using namespace std;

// Constructor by initialisation list : Game
Game::Game()
    : finalImage(0, 0, 600, 600), targetSelected(false), gameInProgress(true)
{
}

// We call the methods to initialise the grid and load the font
void Game::initialize() {
    grid.initialize();
    grid.loadFont();
}

// Method that manages the display based on the progress of the game
void Game::startInterface(SDL_Renderer* pRenderer) {
    if(!targetSelected) {
        grid.drawChoice(pRenderer);
        return;
    }
    if(grid.findNumber(grid.getTarget())) {
        this->gameInProgress = false;
        finalImage.draw(pRenderer, "images/victory.png");
        grid.setFontSize(45);
        grid.drawText(pRenderer, std::to_string(grid.getScore()), 300, 250, {
255, 255, 255, 255 });
    } else if(!grid.findNumber(0) && !grid.shiftAvailable()) {
        this->gameInProgress = false;
        finalImage.draw(pRenderer, "images/defeat.png");
        grid.setFontSize(45);
        grid.drawText(pRenderer, std::to_string(grid.getScore()), 250, 255, {
255, 255, 255, 255 });
    } else {
        grid.draw(pRenderer);
    }
}
```

```
// Method that retrieves the keyboard key and calls the appropriate method
void Game::manageKeyboard(SDL_Event& event) {
    char direction;
    // We retrieve the coordinates of left-click events
    if(!targetSelected) {
        if(event.type == SDL_EVENT_MOUSE_BUTTON_DOWN && event.button.button == SDL_BUTTON_LEFT) {
            grid.choice(event.button.x, event.button.y);
            if(grid.getTarget() != 0) {
                targetSelected = true;
                grid.newGame();
            }
        }
        return;
    }
    if(gameInProgress) {
        switch (event.key.key) {
            case SDLK_UP:
                direction = 'u';
                break;
            case SDLK_DOWN:
                direction = 'd';
                break;
            case SDLK_LEFT:
                direction = 'l';
                break;
            case SDLK_RIGHT:
                direction = 'r';
                break;
            default:
                return;
        }
    } else {
        switch (event.key.key) {
            case SDLK_R:
                grid.newGame();
                this->targetSelected = false;
                this->gameInProgress = true;
                return;
            default:
                return;
        }
    }

    // If a move has been made, a new tile is added
    if (grid.move(direction))
        grid.addTile();
}
```

Fichier main.cpp

```
#include "Game.hpp"
#include "Window.hpp"
#include "GameObject.hpp"
using namespace std;

int main() {

    Window window("Tile Twister", 600, 700);
    window.initialize();

    Game game;
    game.initialize();

    SDL_Event events;
    bool open = true;

    Image duck(600, 578, 150, 150);
    float speed = 125.0f;
    Uint64 last_time = SDL_GetTicks();
```

```
while (open) {
    while (SDL_PollEvent(&events)) {
        switch (events.type) {
            // if you click on the cross, the window closes
            case SDL_EVENT_QUIT:
                open = false;
                break;
            // we listen for other events, including those from the keyboard
            case SDL_EVENT_KEY_DOWN:
                game.manageKeyboard(events);
                break;
            case SDL_EVENT_MOUSE_BUTTON_DOWN:
                game.manageKeyboard(events);
                break;
        }
    }
    Uint64 current_time = SDL_GetTicks();
    float delta_time = (current_time - last_time) / 1000.0f;
    last_time = current_time;
    int value = duck.getCoordinateX() - speed * delta_time;
    duck.setCoordinateX(value);

    if (duck.getCoordinateX() + duck.getSizeX() < 0) {
        duck.setCoordinateX(600);
    }

    SDL_SetRenderDrawColor(window.getRenderer(), 14, 36, 51, 255);
    SDL_RenderClear(window.getRenderer());

    game.startInterface(window.getRenderer());
    duck.draw(window.getRenderer(), "images/duck.png");

    SDL_RenderPresent(window.getRenderer());

    SDL_Delay(16);
}
}
```

03

DEMONSTRATION

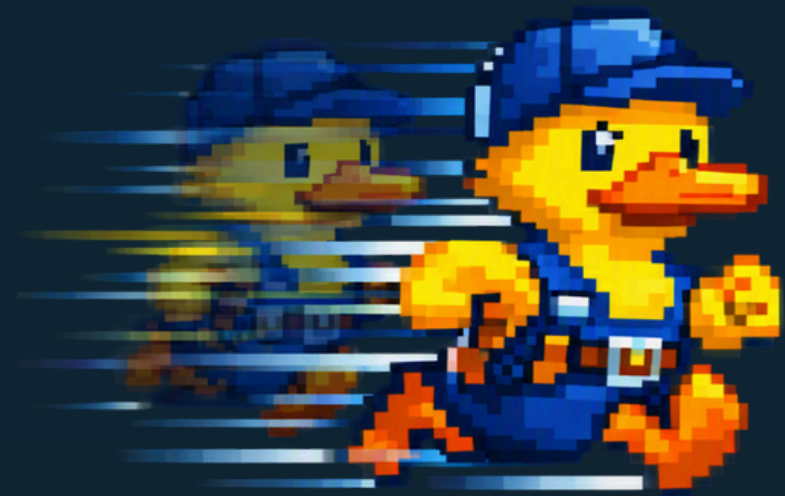


04

CONCLUSION



AMELIORATIONS ENVISAGEES



Ajout d'animations pour rendre les déplacements des tuiles plus fluides



Mise en place de tuiles spéciales pour débloquent des bonus au fil de la partie



Développement d'un système de classement pour comparer les meilleurs scores



DIFFICULTES RENCONTREES



Comprendre le fonctionnement
et les règles du Jeu 2048



Mise en place de l'animation du
Duck Skater



MERCI!



MERCI DE
VOTRE
ATTENTION

MERCI!

